

Bases de datos no relacionales

Práctica Neo4j

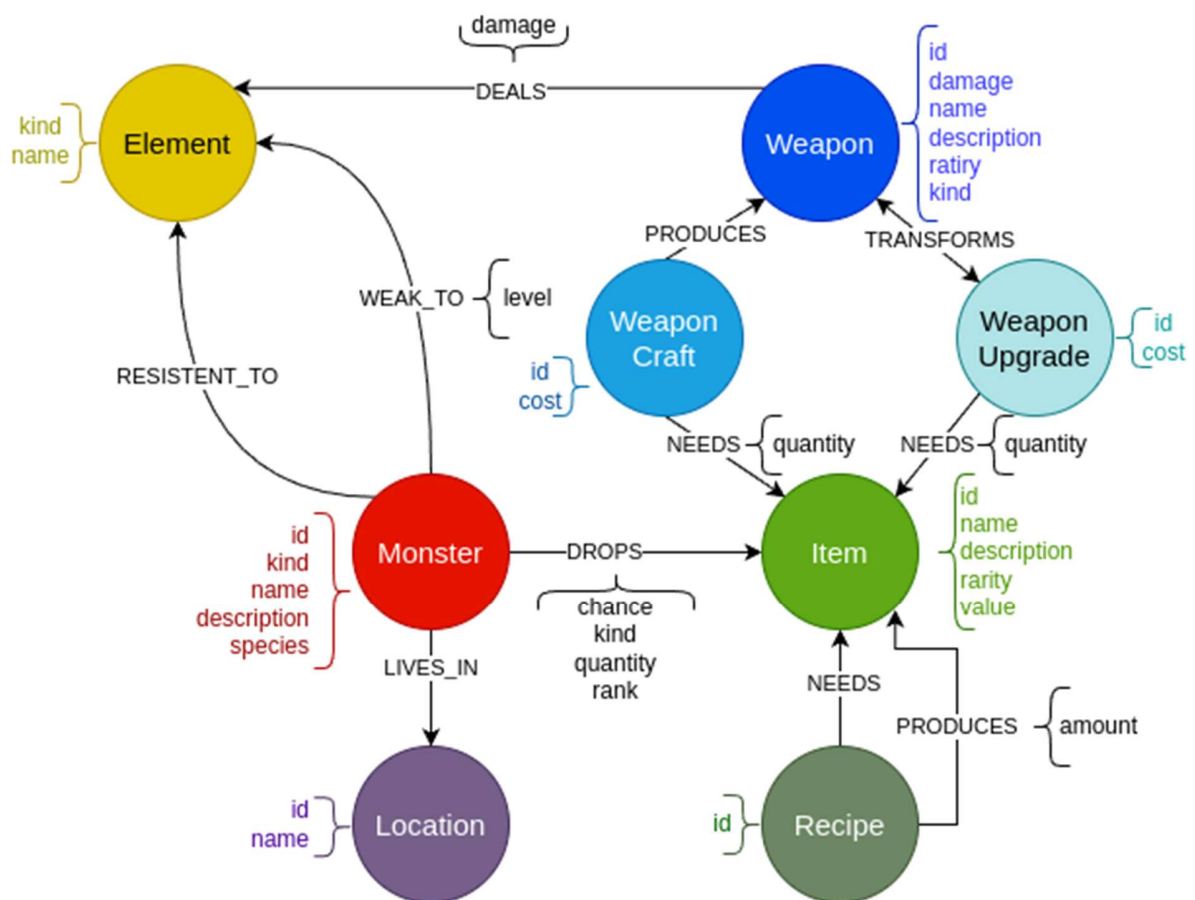


Introducción

La empresa de “Norsewind Studios” tiene implementado un sistema de crafeo bastante complejo en su videojuego “Jotun’s Lair”. El bucle de juego consiste en salir a cazar monstruos que habitan en diferentes biomas y utilizar las partes de estos para crear armas más poderosas. Las armas se pueden crear desde cero o actualizar. Para crear un arma, se necesitan una serie de materiales y una cantidad de dinero. Sin embargo, para actualizar un arma, además del dinero y los materiales, se necesita aportar otra arma, que se usará como base sobre la que realizar la operación de actualización. Ten en cuenta, que las armas se pueden obtener tanto crafeandolas directamente como actualizando otras armas, no obstante, por norma general, crear un arma de cero es más cara que mejorarla. Por último, ten en cuenta que algunas armas solo se pueden obtener mediante actualización y viceversa.

El combate con los monstruos utiliza un sistema de elementos, cada monstruo es débil a ciertos elementos y resistente a otros. Así mismo, cada arma tiene un daño base (sin elemento) y, además, algunas armas son capaces de realizar daño extra de tipo elemental. Dentro de los elementos, hay dos tipos, el daño elemental puro, que puede ser de tipo (fire, water, ice, thunder, dragon o blast), y causa daño extra al monstruo y el daño de estado alterado, que puede ser de tipo (poison, sleep, paralysis o stun), y provoca que el monstruo entre en un estado especial que cambia su comportamiento.

Hasta ahora, toda esta información se guardaba como documentos de diseño, no obstante, el juego ha crecido tanto que la gestión se ha vuelto impracticable y se han decidido a volcar toda la información a una base de datos. Data la naturaleza compleja de los árboles de crafeo, la empresa ha decidido utilizar una base de datos de grafos. El equipo ha creado un pequeño prototipo en neo4j con el siguiente esquema.



Esquema de la base de datos de neo4j

Objetivos

Consultas en la interfaz web

Realiza las siguientes consultas sobre la base de datos usando **la interfaz web**. Para la entrega debes incluir cada consulta en la memoria en formato texto:

1. Devolver el nombre y la descripción de aquellos objetos que no se pueden crear mediante una receta ni se pueden obtener de un monstruo y se utilizan para craftear o mejorar algún arma.
2. El id de receta, nombre de los items usados como input y nombre del item generado como output de las 5 recetas que más ganancias generan. La ganancia que genera una receta es el valor total de los items que genera, menos el coste total de los items

que necesita. Todos los items que se usan en la receta no deben ser producidos por otras recetas.¹

3. El nombre y el tipo del arma que se puede craftear directamente y que más daño hace de tipo fuego. Además, incluir los monstruos que se deben matar para crearla, así como cualquier material extra que se necesario para fabricar el arma y que no se obtiene matando dichos monstruos.
4. Por cada tipo de arma, mostrar el nombre del arma o armas de ese tipo que más daño combinado hace, es decir, la suma de su daño base más todos los daños extra que sean de kind "element". Devolver los tipos ordenados de mayor cantidad a menor cantidad de daño y mostrar el daño que hace dicha arma.
5. El proceso necesario para obtener el arma con id 297. Dicha arma no se puede craftear directamente, solo se puede obtener mediante actualización de otras armas. La consulta debe devolver todas las armas intermedias que debo actualizar desde un arma que sí sea crafteable directamente. Para el arma 297, dicho proceso es [297, 296, 295], es decir, se debe empezar creando el arma 295, luego actualizar el arma 295 a la 296 y por último actualizar la 296 a la 297.
6. El nombre de los monstruos que tengo que matar para fabricar el arma con id 880. Ten en cuenta que dicha arma no se puede craftear directamente, solo se puede obtener mediante actualización de otras armas.
7. El arma con el proceso de fabricación más complejo, es decir, que se necesite pasar por más armas intermedias para llegar a ella desde un arma que es crafteable directamente. Debes devolver, el nombre del arma, el tipo del arma, el nombre de todas las armas por las que debo pasar para llegar a ella y el número de pasos necesarios.
8. El arma con el proceso de fabricación que requiere matar más monstruos diferentes, puedes obviar los empates. Debes devolver el nombre del arma, el nombre de todas las armas por las que debo pasar para llegar a ella y el número de monstruos que es necesario matar.
9. Escribe una consulta que devuelva para cada ubicación (Location), el o los elementos (Element) de tipo "element" frente a los que los monstruos que viven allí son más débiles en conjunto (sumando los niveles de debilidad), y devuelve el nombre de la ubicación, el valor total de esa máxima debilidad y los nombres de dichos elementos.
10. Escribe una consulta que, dado un tipo de arma como parámetro, devuelva qué arma deberías llevar a cada localización. Para ello, se seleccionarán las armas del tipo

¹ Revisa el funcionamiento de la funcion ALL para forzar que todos los elemntos de una lista cumplan una condición <https://neo4j.com/docs/cypher-manual/current/functions/predicate/>

indicado que causen más daño contra los elementos frente a los que, en conjunto, los monstruos de esa localización son más débiles (sumando sus niveles de debilidad). La consulta debe devolver el nombre de la ubicación, los elementos frente a los que los monstruos son más vulnerables y el nombre de las armas recomendadas.

Consultas de análisis de grafos

Realiza las siguientes consultas usando **el cliente python de graph datasciences**. Para entregar debes entregar estas consultas en un cuadernillo jupyter.

Clustering sobre armas

Queremos aplicar un algoritmo de detección de comunidades para agrupar las armas. Consideraremos que dos armas son parecidas si requieren los mismos materiales para su fabricación (no importa si es de creación o actualización).

Paso 1

El primer paso es escribir una consulta que cree una arista nueva entre las armas. Dicha arista se llamará SIMILAR y unirá dos armas si tienen al menos un Item en común entre sus procesos de creación o actualización. La arista tendrá un único atributo llamado weight que contendrá un valor entre 0 y 1. Dicho valor será el índice de Jaccard entre los objetos que se necesitan para construir ambas armas.

Por ejemplo, si un arma requiere estos tres items ["Dragonita", "Cola de Quematrice", "Piedra de fuego"] y otra arma requiere ["Cristal de tierra", "Machalita", "Dragonita"], la arista que una ambas armas tendrá un peso de ($1/5 = 0.2$).

Paso 2

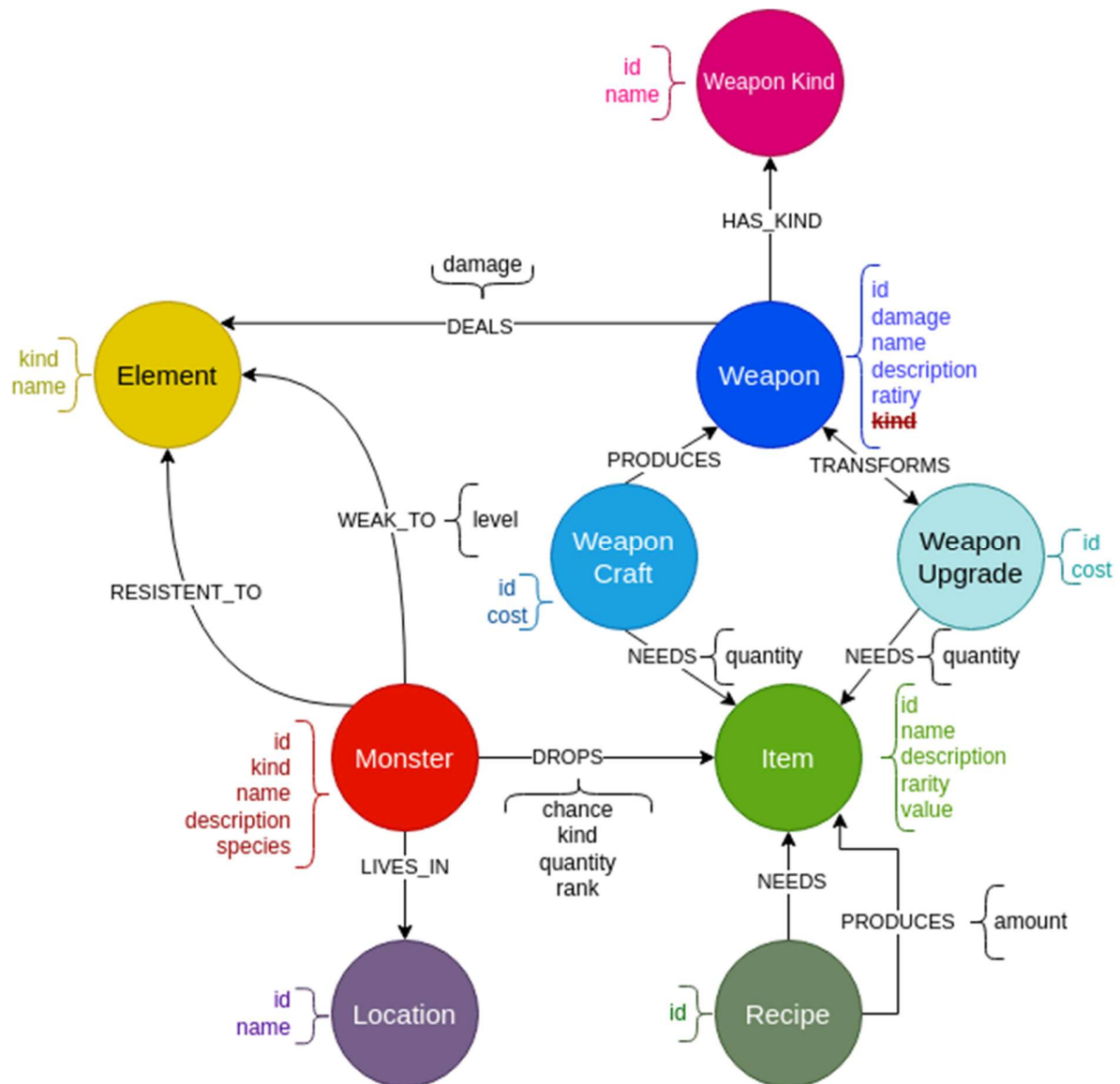
El segundo paso es calcular las comunidades usando el algoritmo de leiden y escribir en cada nodo un atributo que indique a que comunidad pertenece.

Paso 3

El tercer paso es aplicar el algoritmo de pageRank a cada comunidad para obtener la o las armas con más centralidad y escribir por pantalla sus nombres. Este conjunto de armas será el representante de cada comunidad.

Recomendaciones de diseño

Se plantea cambiar el diseño de la base de datos para considerar los tipos de armas como un tipo de nodo en vez de una propiedad. El objetivo de este apartado es responder a la pregunta **¿En qué casos tiene sentido este cambio?** A continuación, tienes el nuevo esquema de la base de datos.



Nuevo esquema base de datos